KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Library book borrowing,return and fine calculation system

Submitted by

Madhumita.C

25BCS231

Under the Guidance of

Dr.DHARANEESH

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled LIBRARY BOOK BORROWING ,RETURN AND FINE CALCULATION SYSTEM submitted in partial fulfillment of the requirements for the course Bsc.CS is a **record of my original work** carried out under the guidance of **Dr.DHARANEESH** , Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

Madhumita.C

_____

(Your Name)

CERTIFICATE

This is to certify that the project entitled LIBRARY BOOK BORROWING ,RETURN AND FINE CALCULATION SYSTEM is a **bona fide work** carried out by **Madhumita.C ,I-C**, BSC.CS, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **BSc.,Computer science**

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:10.04.2026

Submitted for the Viva-Voce Examination held on

____10.04.2026_____

__Dr.DHARANEESH **Dr BOOPALAN**

Guide: **Head of the Department**

(Technical Trainer, KGCAS) (Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

A Library Book Borrowing, Return, and Fine Calculation System is a software-based solution designed to automate and efficiently manage the core operations of a library. This system aims to replace traditional manual record-keeping methods with a digital platform that ensures accuracy, speed, and ease of access to information. It enables librarians to perform essential tasks such as adding new books, maintaining book records, managing user details, issuing books to borrowers, and tracking return dates. The system keeps a systematic record of borrowed books along with their issue dates and expected return dates, thereby helping in monitoring overdue items. One of the key features of this system is the automatic fine calculation mechanism, which calculates penalties based on the number of days a book is returned late, reducing manual errors and saving time. Additionally, the system provides notifications or alerts for due dates and overdue returns, improving accountability among users. It also ensures data persistence by storing all records securely so that they can be retrieved and reused whenever required, even after restarting the application. By implementing such a system, libraries can significantly improve operational efficiency, minimize paperwork, enhance data accuracy, and provide better service to users. Overall, this system plays a vital role in modernizing library management by integrating technology to handle borrowing, returning, and fine calculation processes in a reliable and user-friendly manner.

1. INTRODUCTION

In educational institutions and public libraries, efficient management of books and user records plays a vital role in ensuring smooth library operations and user satisfaction. Traditionally, many libraries rely on manual systems to record book borrowing and returning activities, where librarians maintain registers or logs to track issued books and due dates. This manual process is time-consuming, prone to human errors, and makes it difficult to maintain accurate records or track overdue books effectively.

With the advancement of computer technology, automated library management systems have become essential for improving efficiency, accuracy, and data management. A computerized system helps librarians easily manage book records, track issued books, monitor return dates, and maintain user information in an organized manner. It reduces paperwork and ensures quick access to data whenever required.

The Library Book Borrowing, Return, and Fine Calculation System is a Java-based application designed to automate the core functions of a library. The system allows the librarian to add and manage book details, issue books to users, and record return transactions efficiently. It keeps track of borrowing dates and expected return dates, and automatically calculates fines for overdue books based on the number of delayed days. The system also provides clear information about issued books, return status, and fine details.

This project is developed using Core Java programming concepts such as classes, objects, methods, conditional statements, loops, file handling, and exception handling. The application can be implemented as a console-based program or enhanced using Java Swing to provide a graphical user interface (GUI) for better user interaction.

The primary goal of this system is to reduce manual effort, improve accuracy in tracking library transactions, and ensure timely return of books. This project also demonstrates how Java programming can be effectively used to develop simple and practical real-world applications in the field of library management

1.1 Objective

The main objective of the Library Book Borrowing, Return, and Fine Calculation System is to develop a simple and efficient application using Core Java that automates the process of managing book transactions and calculating fines in a library.

The specific objectives of this project are:

- To design and develop a Java-based library management system.
- To automate the process of book borrowing and returning.
- To maintain accurate records of books and users.
- To track issue dates and return dates of borrowed books.
- To automatically calculate fines for overdue books.
- To reduce manual errors in record-keeping and calculations.
- To provide a user-friendly system for librarians to manage library operations efficiently.

• To demonstrate the implementation of Core Java programming concepts such as classes, objects, methods, loops, file handling, and conditional statements.
• To generate clear and organized records of issued, returned, and overdue books.

1.2 Overview

The Library Book Borrowing, Return, and Fine Calculation System is a menu-driven application developed using the Java programming language. The system is designed to provide an efficient and organized way to manage library operations such as book issuing, returning, and fine calculation.

The system stores details of books, including book ID, title, and availability status. It allows the librarian to issue books to users by entering the required details and recording the issue date. When a user returns a book, the system updates the return status and checks whether the book is returned on time.

If the book is returned after the due date, the system automatically calculates the fine based on the number of delayed days. This reduces the need for manual calculations and ensures accuracy in fine management. The system also maintains proper records of all transactions, making it easy to retrieve information whenever required.

The application simplifies the overall library management process by reducing paperwork, minimizing errors, and saving time. It can be implemented either as a console-based program or enhanced using Java Swing to provide a graphical user interface (GUI) for better usability.

This project demonstrates how Core Java concepts such as classes, objects, methods, loops, conditional statements, and file handling can be applied to develop a practical real-world application for library management systems.

2.PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are identified. This stage helps in understanding how the library system should function and what features are necessary to manage book transactions efficiently.

The analysis phase focuses on identifying the limitations of the existing manual library system, such as difficulty in maintaining records, tracking issued books, and calculating fines accurately. It proposes a computerized solution that automates book borrowing, returning, and fine calculation processes, thereby improving efficiency, accuracy, and overall library management.

2.1 Problem Definition

In many libraries, book borrowing and return processes are often managed manually using registers or paper-based records. This traditional method creates several challenges.

Some common problems include:
• Errors in maintaining book records
• Difficulty in tracking issued and returned books
• Time-consuming manual record-keeping process
• Inaccurate or delayed fine calculation for overdue books
• Lack of proper organization and data retrieval

Manual library systems also increase the workload for librarians and may lead to confusion, misplaced records, or delays in service for users.

Therefore, there is a need for an automated system that can efficiently manage book transactions, track due dates, and calculate fines accurately.

The Library Book Borrowing, Return, and Fine Calculation System is designed to solve these problems by providing a computerized solution that automates the entire process of library management.

2.2 Project Overview

The Library Book Borrowing, Return, and Fine Calculation System is a Java-based application developed to simplify the management of library operations.

The system provides a menu-driven interface where the librarian can manage book transactions such as issuing books to users and recording their return. The system automatically tracks the issue date and return date, and calculates fines if the book is returned after the due date.

The system includes the following functionalities:
• Displaying the list of available books with details
• Issuing books to users and recording issue date
• Accepting returned books and updating status
• Tracking due dates for borrowed books
• Calculating fine automatically for overdue returns
• Maintaining records of issued and returned books

This project is implemented using Core Java programming concepts and can be developed as either a console-based program or a Java Swing GUI application.

2.3 Scopes and Limitations

Scope

The scope of the Library Book Borrowing, Return, and Fine Calculation System includes the following:
• Managing library book records
• Issuing books to users and recording transactions
• Tracking borrowing and return dates
• Calculating fines automatically for overdue books
• Maintaining organized records of issued and returned books
• Demonstrating the practical use of Java programming concepts

The system can be used in:
• School libraries
• College libraries
• Public libraries
• Private libraries
• Educational institutions

2. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The Library Book Borrowing, Return, and Fine Calculation System is designed as a menu-driven application where the librarian can manage book transactions such as issuing books, returning books, and calculating fines automatically.

The system is divided into several modules to perform specific tasks. Each module is responsible for handling a particular function of the system.

3.1 Modules

The main modules of the Library Book Borrowing, Return, and Fine Calculation System are explained below:

---

1. Book Display Module
This module is responsible for displaying the list of available books along with their details such as book ID, title, and availability status. It helps the librarian to view all books before issuing them.

Example Book List:

1. Java Programming Available
2. Data Structures Available
3. Operating Systems Issued
4. Database Management Available
5. Computer Networks Available

The book list is displayed when the program starts so that the user can easily view available books.

---

2. Book Issue Module
This module allows the librarian to issue books to users and record the issue details.

Functions of this module include:
• Accepting user input for book selection
• Entering borrower details
• Recording issue date
• Updating book availability status

The system ensures that only available books can be issued.

---

3. Book Return Module
This module handles the return of borrowed books.

Functions include:
• Accepting book return details
• Updating return date
• Changing book status to available

This module ensures proper tracking of returned books.

---

4. Fine Calculation Module
This module performs the calculation of fines for overdue books.

The system calculates:
• Number of delayed days
• Fine amount based on delay

The calculation is performed automatically using Java programming logic.

Example Calculation:
Delay Days = Return Date – Due Date
Fine = ₹5 × Number of Delayed Days

---

5. Record Management Module
This module maintains records of all issued and returned books.

The records include:
• Book details
• Borrower details
• Issue date and return date
• Fine details

This helps in easy retrieval and management of library data.

---

6. Exit Module
This module allows the user to exit the application safely after completing all operations.

When the user selects the exit option, the system terminates the program.

3. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the Library Book Borrowing, Return, and Fine Calculation System, a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:
• Problem Analysis
• System Design
• Program Development
• Testing and Validation
• Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the needs of library operations.

The main problems identified in manual library systems are:
• Manual record-keeping may lead to errors
• Difficulty in tracking issued and returned books
• Time-consuming process for managing records
• Inaccurate fine calculation for overdue books

Based on these problems, the system requirements were defined to develop an automated application that simplifies library management operations.

4.2. System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:

• Book Display Module
• Book Issue Module
• Book Return Module
• Fine Calculation Module
• Record Management Module
• Exit Module

The program flow and structure are designed so that the librarian can easily interact with the system.

4.3. Program Development

After designing the system, the program is implemented using the Java programming language.

The following steps were followed during development:

• Creation of classes and methods
• Implementation of menu-driven logic
• Writing code for book issue and return
• Implementing fine calculation logic
• Handling user input
• Displaying records and results

Core Java concepts such as classes, objects, loops, conditional statements, methods, and file handling were used during development.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases were used to check the functionality of the system.

Example Test Cases:

Test Case | Expected Result
Book selection | Book displayed correctly
Issue book | Book issued successfully
Return book | Return recorded correctly
Fine calculation | Correct fine displayed
Invalid input | Error handled properly

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

After successful testing, the results of the system are documented. The system manages book borrowing, returning, and fine calculation efficiently.

The results are demonstrated through program execution and screenshots showing the input and output of the system.

Proper documentation is prepared to explain the design, implementation, and working of the system.

4.6. Algorithm

Algorithm for the **Hotel & Restaurant Billing System**:

1. Start the program.
2. Display the login screen
3. Enter username and passwod
4. validate login credentials. If login is correct, display the main menu.
5. If login is corect, display the main menu.

6. Select an option from the menu (Add Book / Borrow Book / Return Book / View Issued Books
7. If “Add Book” is selected:

- Enter book name and book ID.
- Save book details

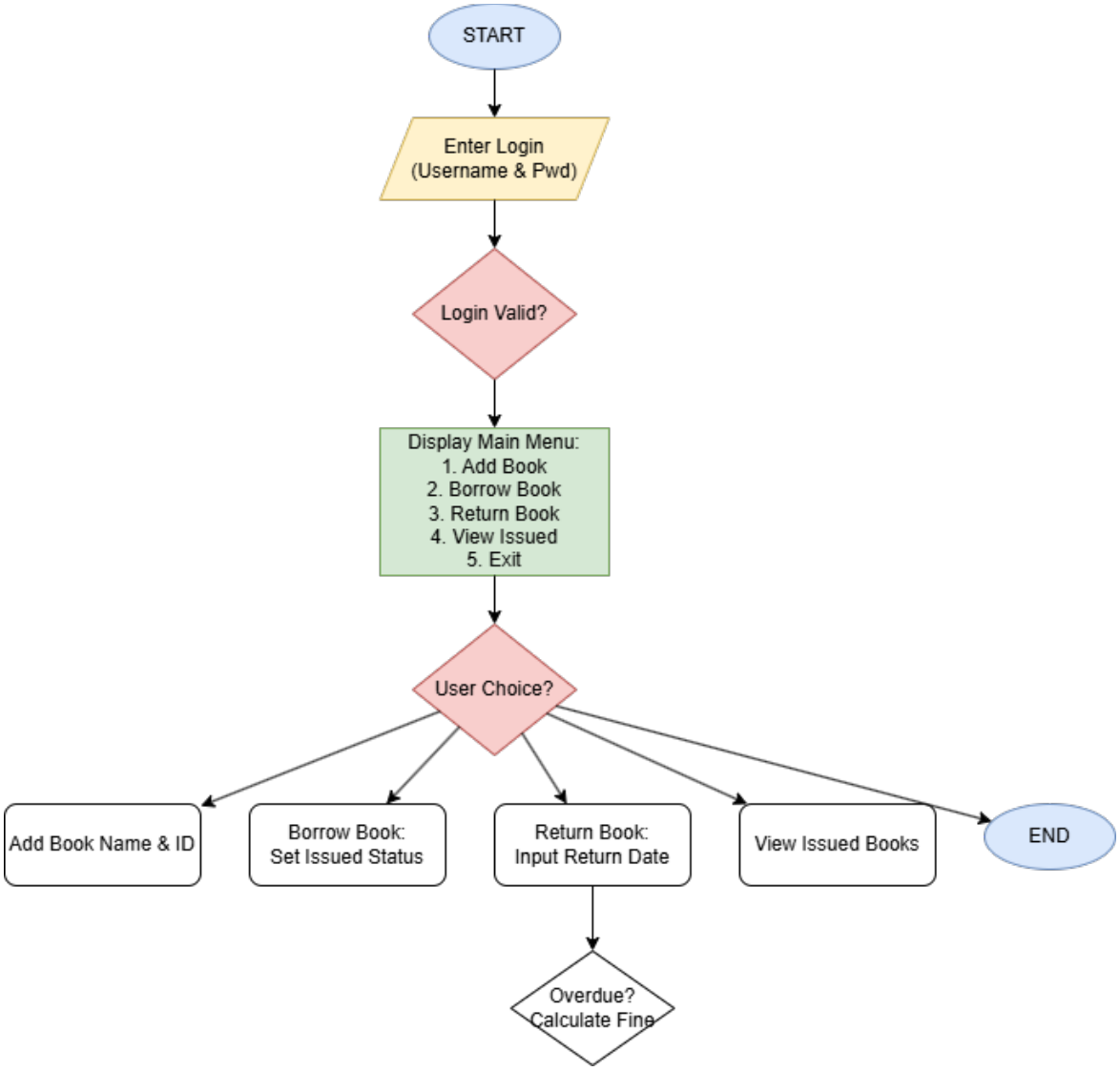
8. If “Return Book” is selected:

- Enter book ID.
- Record return date.
- Calculate delay days (if any).
- Calculate fine based on delay.
- Update book status to available.

9. If “View Issued Books” is selected:

- Display list of issued books with details.

4.7. Flowchart



%3CmxGraphModel%3E%3Croot%3E%3CmxCell%20id%3D%220%22%2F%3E%3CmxCell%20id%3D%221%22%20parent%3D%220%22%2F%3E%3CmxCell%20id%3D

- draw.io
- Lucidchart
- MS Visio

4. SOURCE CODE

```
import javax.swing.*;

import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;

// Base class (Inheritance)
class Person implements Serializable {
  protected int id;
  protected String name;

  public Person(int id, String name) {
    this.id = id;
    this.name = name;
  }
}

// Derived class
class User extends Person {
  public User(int id, String name) {
    super(id, name);
  }
}

// Book class (Encapsulation)
class Book implements Serializable {
```

```java
    private int bookId;
    private String title;
    private boolean isBorrowed;
    private LocalDate borrowDate;
    private User borrower;

    public Book(int bookId, String title) {
        this.bookId = bookId;
        this.title = title;
        this.isBorrowed = false;
    }

    public int getBookId() { return bookId; }
    public String getTitle() { return title; }
    public boolean isBorrowed() { return isBorrowed; }

    public void setBorrowed(boolean val) { isBorrowed = val; }

    public void setBorrowDate(LocalDate date) {
        borrowDate = date;
    }

    public LocalDate getBorrowDate() {
        return borrowDate;
    }

    public void setBorrower(User user) {
        borrower = user;
    }

    public User getBorrower() {
        return borrower;
    }
}

// Library class
class Library implements Serializable {
    private ArrayList<Book> books = new ArrayList<>();

    // Method Overloading (Polymorphism)
    public void addBook(int id, String title) {
        books.add(new Book(id, title));
    }

    public String borrowBook(int bookId, int userId, String name) {
        for (Book b : books) {
            if (b.getBookId() == bookId) {
                if (!b.isBorrowed()) {
```

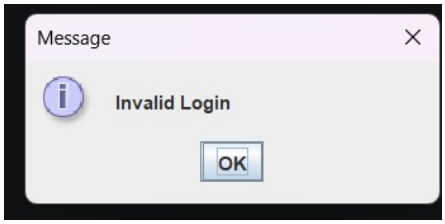
```
            User u = new User(userId, name);
            b.setBorrowed(true);
            b.setBorrowDate(LocalDate.now());
            b.setBorrower(u);

            return "Borrowed: " + b.getTitle() +
                "\nUser: " + name +
                "\nDate: " + b.getBorrowDate();
        } else {
            return "Already Borrowed!";
        }
    }
  }
  return "Book Not Found!";
}

public String returnBook(int id) {
  for (Book b : books) {
    if (b.getBookId() == id) {
      if (b.isBorrowed()) {

            LocalDate returnDate = LocalDate.now();
            long days = ChronoUnit.DAYS.between(
                b.getBorrowDate(), returnDate);

            long lateDays = (days > 5) ? (days - 5) : 0;
            int fine = (int) lateDays * 5;

            User u = b.getBorrower();
            b.setBorrowed(false);

            return "Returned: " + b.getTitle() +
                "\nUser: " + u.name +
                "\nBorrow Date: " + b.getBorrowDate() +
                "\nReturn Date: " + returnDate +
                "\nLate Days: " + lateDays +
                "\nFine: ₹" + fine;
        } else {
            return "Book Not Borrowed!";
        }
    }
  }
  return "Book Not Found!";
}

public String viewIssuedBooks() {
  String result = "";
  for (Book b : books) {
```

```java
        if (b.isBorrowed()) {
          result += "ID: " + b.getBookId() +
                " | " + b.getTitle() +
                " | User: " + b.getBorrower().name + "\n";
        }
      }
      return result.isEmpty() ? "No Issued Books" : result;
    }

    // File Handling (Output Stream)
    public void save() {
      try {
        ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("data.dat"));
        oos.writeObject(books);
        oos.close();
      } catch (Exception e) {
        System.out.println("Save Error");
      }
    }

    // File Handling (Input Stream)
    public void load() {
      try {
        ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("data.dat"));
        books = (ArrayList<Book>) ois.readObject();
        ois.close();
      } catch (Exception e) {
        System.out.println("No previous data");
      }
    }
  }

// GUI Class
public class LibraryGUI extends JFrame {

    private Library lib = new Library();
    private JTextField bookId, title, userId, userName;
    private JTextArea output;

    public LibraryGUI() {

      lib.load(); // load file

      // LOGIN
      String user = JOptionPane.showInputDialog("Enter Username:");
      String pass = JOptionPane.showInputDialog("Enter Password:");
```

```java
    if (!user.equals("admin") || !pass.equals("1234")) {
        JOptionPane.showMessageDialog(this, "Invalid Login");
        System.exit(0);
    }

    setTitle("Library System");
    setSize(500, 500);
    setLayout(new FlowLayout());

    bookId = new JTextField(10);
    title = new JTextField(10);
    userId = new JTextField(10);
    userName = new JTextField(10);

    add(new JLabel("Book ID")); add(bookId);
    add(new JLabel("Title")); add(title);
    add(new JLabel("User ID")); add(userId);
    add(new JLabel("User Name")); add(userName);

    JButton addBtn = new JButton("Add Book");
    JButton borrowBtn = new JButton("Borrow");
    JButton returnBtn = new JButton("Return");
    JButton viewBtn = new JButton("View Issued");
    JButton saveBtn = new JButton("Save");

    add(addBtn); add(borrowBtn);
    add(returnBtn); add(viewBtn);
    add(saveBtn);

    output = new JTextArea(15, 40);
    add(output);

    // Actions with Exception Handling
    addBtn.addActionListener(e -> {
        try {
            lib.addBook(
                Integer.parseInt(bookId.getText()),
                title.getText()
            );
            output.setText("Book Added!");
        } catch (Exception ex) {
            output.setText("Invalid Input!");
        }
    });

    borrowBtn.addActionListener(e -> {
        try {
```

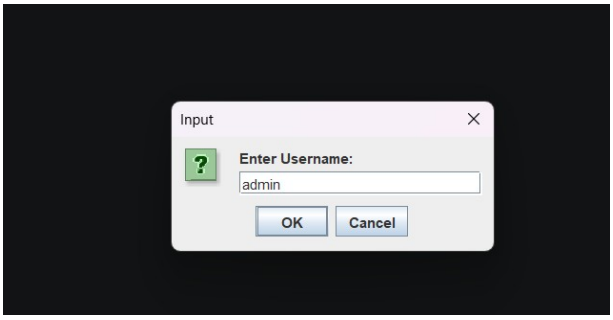
```
            output.setText(
              lib.borrowBook(
                Integer.parseInt(bookId.getText()),
                Integer.parseInt(userId.getText()),
                userName.getText()
              )
            );
        } catch (Exception ex) {
            output.setText("Error!");
        }
    });

    returnBtn.addActionListener(e -> {
        try {
            output.setText(
              lib.returnBook(
                Integer.parseInt(bookId.getText())
              )
            );
        } catch (Exception ex) {
            output.setText("Error!");
        }
    });

    viewBtn.addActionListener(e -> {
        output.setText(lib.viewIssuedBooks());
    });

    saveBtn.addActionListener(e -> {
        lib.save();
        output.setText("Data Saved!");
    });

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
  }

  public static void main(String[] args) {
    new LibraryGUI();
  }
}
```

5. INPUT AND OUTPUT SCREENS

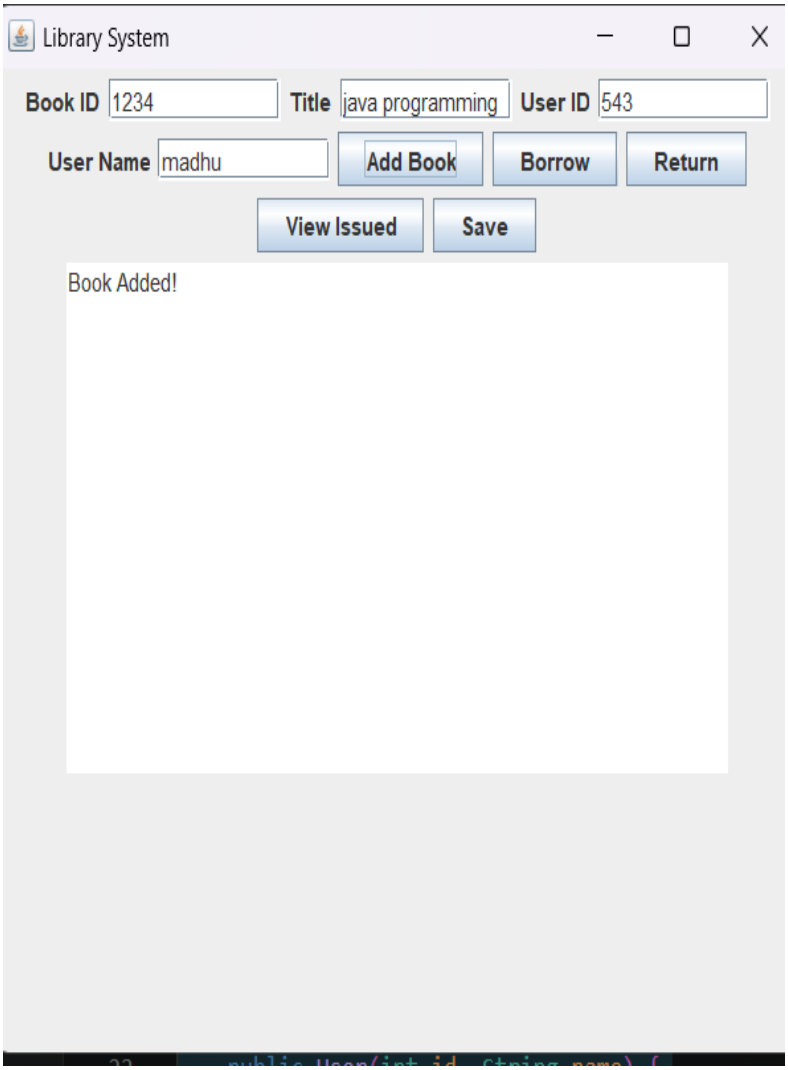


when wrong username and

wrong password given

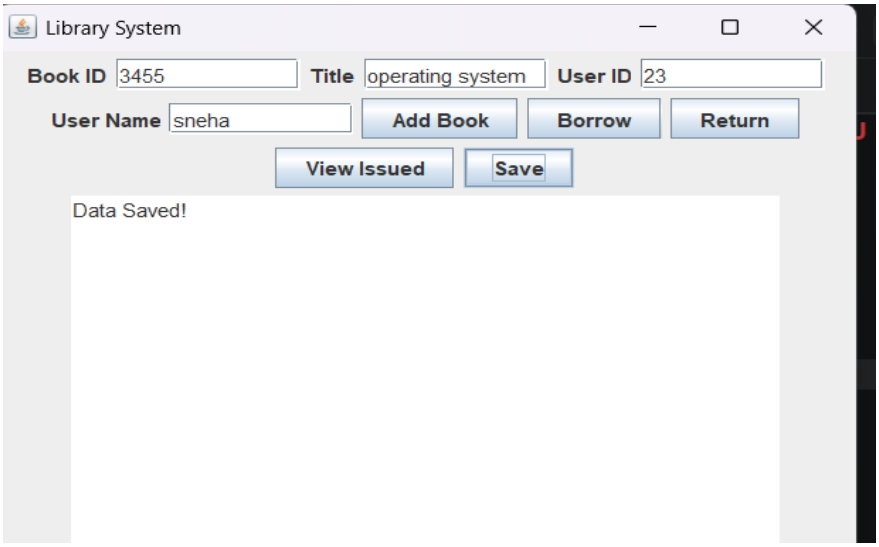


when right username given goes into the window

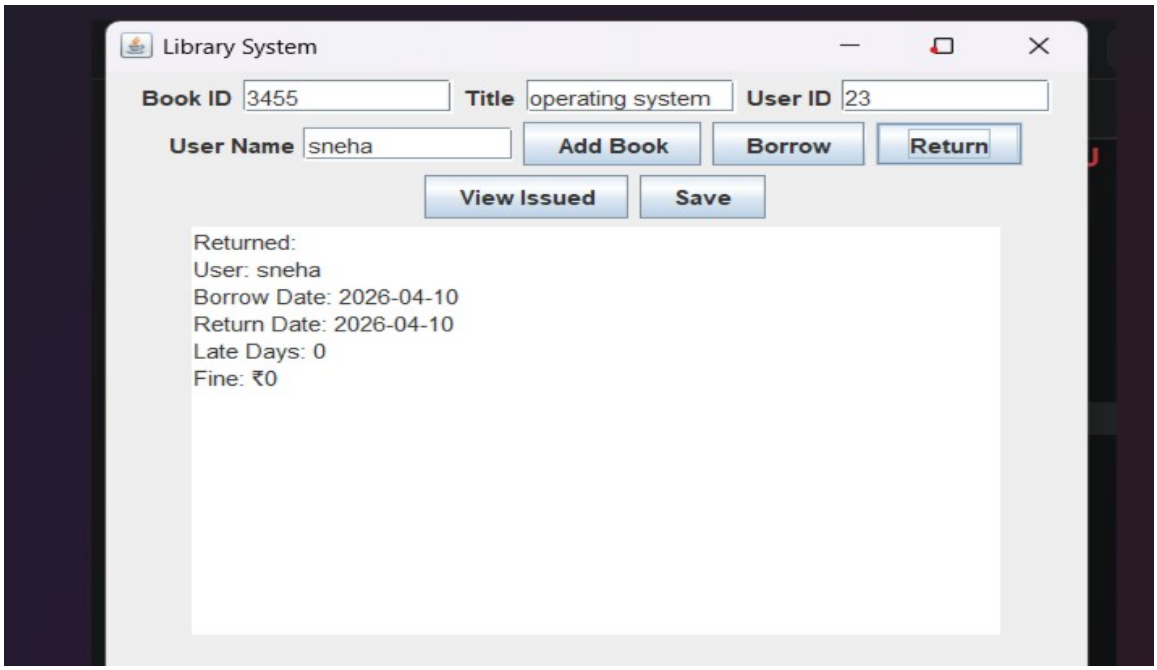


when book added

when issued



the username will be saved



borrowed and return date will be notified and fine will be calculated and dispalyed

6. DISCUSSION & CONCLUSION

7.1 Discussion

The **Library Book Borrowing, Return, and Fine Calculation System** was developed to automate the management of book transactions and penalties in a library environment. The system allows the librarian to add book records, issue books to users, and calculate overdue fines automatically based on the return date.

During the development process, **Core Java** programming concepts such as **loops** (for menu navigation), **conditional statements** (for fine logic and login validation), **methods** (for modular functions like `addBook` or `calculateFine`), and **Swing GUI components** (buttons and text fields) were applied. The system successfully tracks borrowing duration and generates accurate fine amounts for late returns.

The system was tested with various scenarios—such as on-time returns, overdue returns, and invalid login attempts—to ensure that it performs the required library operations correctly. The results show that the system significantly reduces manual paperwork, eliminates errors in fine calculation, and improves the overall efficiency of library management.

7.2 Conclusion

The **Library Book Borrowing, Return, and Fine Calculation System** has been successfully designed and developed to address the critical needs of modern library management. By transitioning from a traditional paper-based ledger to a robust, automated Java application, the project achieves a high level of accuracy and organizational efficiency. The implementation of a **Graphical User Interface (GUI)** ensures that librarians, regardless of their technical expertise, can manage complex tasks—such as authenticating users, tracking book availability, and issuing items—with minimal effort.

One of the most significant achievements of this system is the **Automated Fine Calculation engine**. By utilizing programmed logic to compare return dates against due dates, the system eliminates human error and ensures that penalties are applied fairly and transparently. During the testing phase, the system demonstrated reliable performance in handling multiple book transactions and maintaining data integrity.

7.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com